



spriep01@estudiantes.unileon.es

Final Report Quantization Techinques Internship

Saioa Prieto Portugal

Abstract

This report summarizes the work carried out during my internship in the context of gender and age prediction models. The main objective of the internship was to study and evaluate different optimization, quantization and compression techniques and their application to the MiVOLO-D1 model. The work involved the study of the relevant quantization techniques as PTQ and Knowledge Distillation, the implementation and testing of different approaches, and the analysis of their performance on both UTK and MORPH II datasets. This report presents the methodology followed, the main results obtained, and the conclusions drawn from the work.

*All experiments, benchmarking iterations, and training pipelines reported in this study were executed under the following local hardware and software specifications:

- Operating System: Windows 11
- CPU: AMD Ryzen 7 5800H with Radeon Graphics
- System RAM: 16 GB
- Integrated GPU: AMD Radeon Graphics
- Discrete GPU: NVIDIA GeForce RTX 3050

León, Spain
September 8, 2026

Contents

1	Theoretical Foundations of Model Quantization and Optimization	3
1.1	Quantization	3
1.1.1	Post-Training Quantization (PTQ)	3
1.1.2	Quantization-Aware Training (QAT)	3
1.1.3	Sub-8-Bit Quantization	4
1.2	Model Compression	4
1.2.1	Knowledge Distillation (KD)	4
1.2.2	Hardware-Aware Structured Pruning	5
1.2.3	Parameter-Efficient Fine-Tuning (PEFT)	5
1.3	Inference Optimization	6
1.3.1	Dynamic Routing and Early Exiting	6
1.3.2	Graph-Level Optimization and Compilation	6
2	The Baseline Architecture: MiVOLO-D1	7
2.1	Official Pre-trained Weights Evaluation	7
2.2	Selection of the Primary Baseline Model	8
3	Benchmark Datasets and Preprocessing Pipeline	8
3.1	MORPH2 Face Preprocessing Pipeline via RetinaFace	8
3.2	Performance Metrics and Evaluation Protocol	9
4	Post-Training Quantization in FP16 (PTQ FP16)	10
4.1	Performance Results	10
4.2	Performance Analysis and Findings	11
5	Post-Training Quantization in INT8	12
5.1	Native CUDA Limitations	12
5.2	Compilation Incompatibilities in ONNX and TensorRT	12
5.3	Evaluation of Alternative PyTorch Quantization Frameworks	13
6	Knowledge Distillation	14
6.1	Student Architectures	14
6.2	Data Organization and Logit Caching	14
6.3	Two-Phase Training Protocol	15
6.4	Performance Results	16
6.5	Performance Analysis and Key Findings	18
7	Future work	19

1 Theoretical Foundations of Model Quantization and Optimization

1.1 Quantization

Model quantization constitutes a fundamental paradigm in neural network compression, aimed at mitigating memory bandwidth bottlenecks, reducing physical storage footprints, and accelerating inference execution without requiring structural architectural changes. Formally, quantization involves mapping high-precision numerical representations—typically 32-bit floating-point format (FP32)—to lower-precision data types, such as 16-bit floating-point (FP16), 8-bit integers (INT8), or sub-8-bit formats (INT4 / W4A8).

In the context of deep vision architectures and regression pipelines, two primary methodologies govern quantization strategies:

1.1.1 POST-TRAINING QUANTIZATION (PTQ)

PTQ is applied directly to a pre-trained model without requiring access to the original training pipeline or iterative weight retraining. By converting weights (and optionally activations) into lower-precision formats post-convergence, PTQ offers an immediate optimization pathway. It is generally categorized into:

- **Dynamic Quantization:** where activation ranges are determined at runtime, incurring some computational overhead but simplifying application (targeting weights of operations like Conv/MatMul)
- **Static Quantization:** Activation ranges and quantization parameters (scales and zero-points) are pre-determined using a representative calibration dataset prior to deployment.

While attractive for immediate deployment, traditional PTQ can induce accuracy degradation, particularly in sensitive architectures like depthwise convolutional neural networks

1.1.2 QUANTIZATION-AWARE TRAINING (QAT)

When PTQ excessively impairs the model accuracy (drops greater than 1%), or really small formats want to be used (less than 8bit) or dealing with sensitive architectures as depth wise CNNs, QAT is used. QAT can yield higher accuracy by simulating quantization during training, but it implies the training of the model which PTQ doesn't.

When the CNN performs the forward pass, the framework (such as PyTorch) activates special modules called "simulated quantizers," which take the high-precision

weights (FP32) and temporarily transform them into low precision weights (INT8 or INT4). Since the network is making errors due to this lack of decimal places, during the backward pass, the model slightly adjusts its original high-precision parameters to compensate for the error introduced by the low precision. At the end of the process, the model is exported, transforming the corrected weights into lighter formats.

Modern QAT variants mitigate training instabilities (such as weight oscillations via oscillation-robust mechanisms) and can match or even surpass full-precision baselines, especially in aggressive low-bit regimes.

1.1.3 SUB-8-BIT QUANTIZATION

Crossing the traditional 8-bit barrier involves mixed-precision formats and sub-8-bit strategies (such as W4A8, denoting 4-bit weights and 8-bit activations). Across recent work, W8A8 is effectively “safe” for image and video models, and W4A8 is now feasible for diffusion transformers, attention, and several vision backbones with carefully designed PTQ/QAT.

1.2 Model Compression

1.2.1 KNOWLEDGE DISTILLATION (KD)

Knowledge Distillation (KD) is a model compression paradigm aimed at transferring the generalization capabilities, feature representation capacity, and predictive accuracy of a large, high-capacity "teacher" network into a compact, lightweight "student" network. Unlike architectural pruning or standard post-training quantization, KD allows for significant reductions in parameter count and computational complexity without necessitating structural modifications to the target deployment architecture. Traditionally, KD relies on two primary channels of information transfer from the teacher to the student:

- Soft Probabilities / Logits: Classic KD leverages the teacher’s softmax probabilities (often modulated via temperature scaling) as "soft labels" alongside ground-truth loss, allowing the student to learn rich class inter-relationships. Advanced methodologies extend this by employing stage-wise logit distillation (comparing outputs across intermediate classifier stages). Furthermore, using only the teacher logits enables **data-free knowledge distillation**, ensuring strict data privacy and confidentiality by generating a representative surrogate dataset directly from the model’s logits.
- Intermediate Feature Maps: Beyond final outputs, feature-based distillation enables the student network to inspect and mimic how the teacher’s internal layers process spatial and semantic information. One of the primary hurdles

is addressing architectural mismatch, for which dedicated techniques have been specifically designed.

1.2.2 HARDWARE-AWARE STRUCTURED PRUNING

Pruning aims to eliminate uninformative or low-magnitude weights from a neural network to compress the model and accelerate execution. Unlike unstructured pruning—which populates weight tensors with irregular zero entries and introduces significant memory bandwidth bottlenecks on modern hardware—hardware-aware structured pruning enforces rigid, predictable sparsity patterns natively aligned with compute engines.

A prominent formulation is $N:M$ fine-grained structured sparsity, in particular the 2:4 sparsity pattern supported by NVIDIA Tensor Cores. Under this scheme, every contiguous group of $M = 4$ weights is constrained to contain exactly $N = 2$ non-zero values. This enables dense matrix multiplications to run at up to $2\times$ the operational throughput without the routing overhead of sparse formats. To train models directly into valid 2:4 sparse representations while preserving downstream accuracy, several specialized strategies have emerged:

- SR-STE (Sparse-Refined Straight-Through Estimator)
- S-STE (Continuous 2:4 Pruning)
- AdaPrune and Tilewise / Tile-Vector-Wise (TVW) Pruning

1.2.3 PARAMETER-EFFICIENT FINE-TUNING (PEFT)

PEFT consists of methods that adapt models, particularly LLMs, by freezing most of the original model’s parameters and fine-tuning only a small subset of them.

- LoRA: It freezes the original weight matrix (W), creates two much smaller matrices A and B which will be the ones that are trained. Then user has the option to merge all 3 matrices to reduce latency but loose adaptability to new activities, or leaving all 3 matrices separated and merging both results (one from using W and another from using A and B) at the end of the cycle.
- QLoRA (Quantized LoRA): Compress the original frozen base model to 4-bit (PTQ or QAT based), and add the LoRA adapters (A, B) in normal precision (16-bit). This eliminates the option of merging the matrices, SOTA techniques try to solve this latency gap.

1.3 Inference Optimization

1.3.1 DYNAMIC ROUTING AND EARLY EXITING

Dynamic inference enables networks to adaptively modulate compute depth based on input complexity, avoiding exhaustive passes over deep architectures for easily resolvable samples.

Early-Exit Architectures

On Early-Exit CNNs (EENets) intermediate exit branches are added alongside the primary backbone. Each exit incorporates a classifier paired with a learnable confidence estimator. When intermediate confidence surpasses a pre-calibrated threshold, execution halts and returns the intermediate prediction; otherwise, the sample continues downstream.

Alternatively, Metric-Learning Early Exits are tailored for verification pipelines where classification heads are absent, these utilize intermediate representations (e.g., Bag-of-Features) coupled with shallow linear regressors to approximate terminal embedding projections.

Dynamic-Depth Architectures

Inserts compact, trainable gating routers across attention and multi-layer perceptron (MLP) layers. Inputs scoring below threshold values bypass corresponding layers, yielding substantial execution speedups. In facial analysis tasks, early-exit strategies pose distinct fairness challenges. Intermediate representations amplify matching **divergences across demographic groups**, models tend to assign higher confidence earlier to dominant demographic groups, routing them through shallower exits with higher marginal error risks, while underrepresented groups traverse the entire network depth.

1.3.2 GRAPH-LEVEL OPTIMIZATION AND COMPILATION

Deep learning frameworks represent network topologies as directed Computation Graphs, where graph nodes correspond to discrete mathematical kernels and edges represent continuous tensor dataflows. Graph compilers (such as TensorRT, ONNX Runtime, and specialized research compilers) analyze and restructure these graphs prior to hardware execution.

Standard execution requires intermediate tensor buffers to transition continuously between off-chip high-bandwidth memory (HBM/DRAM) and fast on-chip memory (SRAM/registers) across adjacent layers. Operator fusion merges consecutive operational nodes into unified execution kernels—such as fusing multi-

layer chains like Convolution $\rightarrow$ Batch Normalization $\rightarrow$ ReLU or batched MatMul $\rightarrow$ Softmax blocks—drastically lowering memory traffic and operational kernel launches. Compilers such as Hidet demonstrate automated post-scheduling fusion that integrates operations like img2col into unified kernels, outpacing conventional execution runtimes.

Compilers systematically run graph cleanup passes, including constant folding, dead-code elimination, common subexpression elimination, and algebraic simplification. Advanced graph compilers (e.g., Forge-UGC, GTCO, and AutoGTCO) execute multi-pass automated search strategies across tensor expression spaces, eliminating redundant transitions between processors and outperforming static runtime graphs by 18% to 73% in end-to-end latency.

2 The Baseline Architecture: MiVOLO-D1

The primary baseline employed in this research is MiVOLO-D1, an advanced multi-task deep neural network designed for facial age and gender estimation. Architecturally, MiVOLO is constructed on top of the **timm library** and integrates hybrid vision mechanisms—including attention blocks, PatchEmbed components, and normalization layers alongside standard linear projections.

While the full MiVOLO framework (such as the MiVOLO-D2 variant) possesses the capacity to ingest both a face crop and an additional body crop to exploit contextual visual cues, **the experimental scope of this study was strictly constrained to face crops only.**

Consequently, before inference, input images are resized to the network’s expected input dimensions (224×224 pixels) and normalized according to standard image statistics with PyTorch’s normalization pipeline. The estimator outputs three target predictions: continuous predicted age, gender confidence score, and binary gender classification.

2.1 Official Pre-trained Weights Evaluation

To establish an empirical baseline before initiating model compression, multiple official pre-trained weight configurations available in the MiVOLO repository were evaluated across the UTKFace and MORPH2 benchmarks under identical FP32 precision and single-image execution (batch size = 1).

1. model_only_age_imdb_4.29.pth.tar (Age-Only, IMDB-trained)
2. model_utk_age_gender_4.23_97.69.pth.tar (Age & Gender, UTK-trained)
3. model_imdb_age_gender_4.22.pth.tar (Age & Gender, IMDB-trained)

Table 1: Performance comparison of the evaluated models on UTKFace and MORPH2.

Model	Dataset	MAE (years)	RMSE	Gender Accuracy (%)	FPS
model_only_age_imdb_4.29.pth.tar	UTKFace	5.52	7.38	–	41.26
	MORPH2	4.90	6.05	–	42.14
model_utk_age_gender_4.23_97.69.pth.tar	UTKFace	8.10	11.99	90.95	42.74
	MORPH2	5.40	6.55	97.24	41.83
model_imdb_age_gender_4.22.pth.tar	UTKFace	5.33	7.18	92.13	41.46
	MORPH2	4.46	5.60	97.28	41.84

2.2 Selection of the Primary Baseline Model

Based on these comparative benchmark evaluations, model_imdb_age_gender_4.22.pth.tar was selected as the definitive baseline checkpoint for all subsequent optimization, quantization (PTQ FP16/INT8), and knowledge distillation experiments. It exhibits a baseline model size of 98.70 MB and a peak VRAM consumption of 115.77 MB during single-batch CUDA execution.

3 Benchmark Datasets and Preprocessing Pipeline

To rigorously evaluate model accuracy, computational throughput, and hardware efficiency across both full-precision baselines and compression strategies, two standard benchmark datasets were utilized.

UTKFace: A widely recognized facial dataset spanning diverse age, ethnicity, and gender distributions. Across the benchmark iterations, a total of **23,708** valid face images were evaluated.

MORPH2: A large-scale longitudinal facial database extensively used for age progression and demographic estimation. From an initial pool of 55,608 images, a rigorous pre-filtering step removed unprocessable samples, yielding a final benchmark test split of **54,932** valid face images.

3.1 MORPH2 Face Preprocessing Pipeline via RetinaFace

Because MiVOLO-D1 in this study was evaluated strictly in a face-only configuration—disregarding body bounding boxes—a dedicated facial extraction pipeline was executed prior to model inference. The entire MORPH2 dataset (55,608 raw images) was processed using **RetinaFace** to detect and crop the primary facial regions. During this preprocessing phase, a total of 676 images were discarded:

- 246 images were identified as invalid samples (i.e., corrupt files or images containing no human facial structures)

- 430 images corresponded to faces that could not be reliably detected by RetinaFace.

Removing these invalid entries resulted in the definitive split of 54,932 high-quality face crops used throughout all MORPH2 benchmarks.

3.2 Performance Metrics and Evaluation Protocol

To comprehensively evaluate predictive accuracy, computational efficiency, and deployment requirements, the evaluation framework considers a set of complementary performance metrics. All benchmarking experiments were executed with a batch size of 1 (processing each image individually to reflect edge deployment constraints) and metrics were averaged over several iterations to guarantee statistical reliability.

Age Regression Metrics.

The **Mean Absolute Error (MAE)** is used as the primary metric for age regression. It measures the average absolute difference, in years, between the predicted age and the chronological ground truth:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|, \quad (1)$$

where y_i denotes the ground-truth age, $\hat{y}_i$ denotes the predicted age, and N is the number of evaluated samples.

The **Root Mean Squared Error (RMSE)** is reported as a complementary regression metric. Unlike MAE, RMSE assigns greater weight to larger prediction errors:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}. \quad (2)$$

Gender Classification Metrics.

For models performing gender prediction, classification performance is measured using **gender accuracy**, defined as the proportion of correctly classified samples over the total number of evaluated samples. In addition, **confusion matrices are reported** to provide a detailed characterization of true positives, true negatives, false positives, and false negatives for the two gender classes.

Inference Performance.

Throughput is reported in **Frames Per Second (FPS)**, representing the number of images processed per second during inference. In addition, the **total inference time** required to process each complete dataset split is recorded to characterize the overall computational cost of evaluation.

Hardware and Storage Requirements.

Hardware resource consumption is characterized by peak GPU memory allocation and model size. **Peak VRAM** corresponds to the maximum CUDA memory allocated during inference, reported in MB. **Model size** represents the physical storage footprint of the corresponding model checkpoint on disk, also measured in MB.

4 Post-Training Quantization in FP16 (PTQ FP16)

To bypass the excessive computational cost and prolonged retraining schedules required by Quantization-Aware Training (QAT), Post-Training Quantization (PTQ) was deployed as the initial compression pathway. An architectural inspection of the MiVOLO-D1 baseline reveals that linear layers constitute 97.37% of the total network parameters. To maximize hardware speedup while preventing numerical instability and predictive degradation, quantization was applied only to linear layers.

4.1 Performance Results

The selectively quantized FP16 model was benchmarked against the full-precision baseline across both datasets. From an implementation standpoint, applying this half-precision representation does not require complex third-party tools. The official MiVOLO baseline provides native support for FP16 quantization directly through its inference pipeline: when initializing the estimator or invoking the inference call, an internal execution parameter converts and casts all the relevant model weights to FP16 seamlessly on the target CUDA device.

Table 2: Performance comparison of MiVOLO-D1 baseline versus Dynamic FP16 across UTKFace and MORPH II datasets (100%).

Metric	UTKFace		MORPH II	
	Baseline (FP32)	Dynamic FP16	Baseline (FP32)	Dynamic FP16
MAE (years) ↓	5.33	5.33	4.86	4.46
RMSE ↓	7.18	7.18	6.09	5.59
FPS ↑	41.46	45.41	36.55	44.60
Total Inference Time (s) ↓	571.91	522.15	1521.53	1221.44
Peak VRAM (MB) ↓	115.77	63.98	115.77	63.98
Model Size (MB) ↓	98.67	49.33	98.70	49.93
Gender Accuracy (%) ↑	92.13%	92.15%	97.10%	97.29%

4.2 Performance Analysis and Findings

The empirical evaluation reveals substantial hardware optimizations accompanied by robust predictive stability across both benchmarks. In terms of memory footprint, selective **half-precision quantization achieved an exact 50.0% reduction** in physical storage on disk, halving the model checkpoint from 98.67 MB to 49.33 MB. This storage compression is mirrored by runtime GPU memory efficiency, where **peak VRAM allocation dropped by 44.73% during inference**, descending from 115.77 MB down to 63.98 MB.

Regarding computational throughput, transitioning to dynamic FP16 delivered notable acceleration across both benchmark datasets on the target CUDA architecture. On the UTKFace dataset, inference throughput increased from 41.46 FPS to 45.41 FPS which **reduced the total inference time an 8.70%**. The acceleration was even more substantial on the MORPH II benchmark, where throughput rose from 36.55 FPS to 44.60 FPS, achieving a **19.72% reduction in end-to-end evaluation time**.

On the UTKFace dataset, continuous age regression metrics remained completely invariant, maintaining a baseline MAE of 5.33 years and an RMSE of 7.18, while binary **gender classification showed a negligible gain of +0.02%** (from 92.13% to 92.15%). On the MORPH II benchmark, dynamic FP16 not only preserved fidelity but improved overall precision, reducing the MAE from 4.86 to 4.46 years (an **8.23% error decrease**), lowering the RMSE from 6.09 to 5.59, and **increasing gender classification accuracy** from 97.10% to 97.29%.

While counterintuitive, this slight performance gain following half-precision conversion is widely documented in deep learning compression literature. Over-parameterized neural architectures are prone to overfitting to high-frequency training noise when operating in 32-bit floating-point precision. Reducing numerical precision to FP16 acts as an implicit regularization mechanism by introducing small rounding perturbations to weight representations, which attenuates noise

sensitivity and smooths the loss landscape

Consequently, dynamic FP16 serves as an optimal deployment strategy for MiVOLO-D1, simultaneously delivering faster inference throughput, halving model weights, and sustaining predictive fidelity.

5 Post-Training Quantization in INT8

5.1 Native CUDA Limitations

Following the evaluation of FP16 precision, 8-bit integer Post-Training Quantization (PTQ INT8) was explored to achieve higher memory compression and throughput. However, native execution under PyTorch on CUDA encounters fundamental architectural limitations.

Standard PyTorch backends supporting INT8 computation (such as FBGEMM and QNNPACK) are explicitly engineered and optimized for x86 and ARM CPU architectures rather than GPU acceleration.

CUDA lacks an out-of-the-box native INT8 execution kernel within standard PyTorch execution paths. Consequently, although model weights can be physically stored in INT8 precision in memory, they are dynamically dequantized back to FP32 or FP16 during inference immediately before matrix operations are computed.

Rather than accelerating throughput, this continuous runtime dequantization introduces severe computational overhead, ultimately resulting in slower end-to-end inference compared to native floating-point execution.

5.2 Compilation Incompatibilities in ONNX and TensorRT

To circumvent the runtime dequantization penalty on CUDA and unlock true 8-bit acceleration, industry-standard deployment relies on static inference compilers such as NVIDIA TensorRT, which offer dedicated INT8 kernels. TensorRT requires a pre-compiled, static computation graph, which is traditionally obtained by exporting the PyTorch model to the Open Neural Network Exchange (ONNX) format. When attempting to export MiVOLO-D1, this compilation pipeline failed due to structural characteristics of the underlying architecture:

- **Dynamic Execution and State Modification:** MiVOLO is built on top of the timm library and relies on custom architectural modules that dynamically modify their internal states during runtime execution. This rules out the possibility of constructing the fixed, static computation graph required by standard exporter routines.

- **Unsupported Graph Operators:** The export process fails on specific dynamic tensor operations embedded within the model—most notably fold and unfold operations (as col2im) present in custom spatial manipulation layers.

Because of this inherent dynamic behavior, modern deployment backends—including TensorRT and Intel OpenVINO—fail to parse or compile these custom layers, as documented across their official issue trackers and execution logs.

5.3 Evaluation of Alternative PyTorch Quantization Frameworks

To determine whether INT8 quantization could be realized purely within Python without relying on ONNX or TensorRT export pipelines, three contemporary PyTorch-centric libraries were systematically evaluated.

Optimum.quantize

This framework replaces standard parameters with custom quantized types (QBytes). However, because MiVOLO depends internally on timm components that expect standard PyTorch floating-point Tensor structures, the integration triggered persistent type mismatch errors during forward passes.

torch.ao

PyTorch’s legacy Architecture Optimization module (torch.ao) was evaluated but discarded due to its lack of direct CUDA execution support for standard INT8 quantization workflows.

torchao.quantization

The modern torchao library served as the only framework capable of executing dynamic INT8 workflows without immediate runtime exceptions. Under this approach, layers were quantized using dynamic activation and weight configs (e.g., Int8WeightOnlyConfig, Int8DynamicActivationInt8WeightConfig, and AffineQuantizedTensor). Inspection confirmed that target modules—such as post_network.1.mlp.fc2—were successfully converted into affine-quantized structures. Despite avoiding runtime crashes, profiling revealed that the internal execution methods in MiVOLO continued to recast quantized tensors back to FP32 prior to GPU kernel execution. As a result, the model suffered from the same runtime dequantization bottlenecks observed in native PyTorch.

6 Knowledge Distillation

Given that PTQ INT8 produced no latency improvements due to CUDA dequantization penalties, and that structural graph compilation was fundamentally blocked by MiVOLO’s dynamic timm layers, this optimization line was discontinued. Rather than altering the original architecture through destructive refactoring, research shifted toward Knowledge Distillation (KD). This approach bypasses the dynamic constraints of MiVOLO by using the full-precision MiVOLO-D1 as a static "teacher" to supervise compact, export-friendly student networks designed for seamless deployment.

6.1 Student Architectures

To replace the dynamic operator graph of MiVOLO with hardware-friendly topologies, student networks were constructed using the timm library within a unified static wrapper class (StaticStudentMiVOLO).

The selection of student architectures was fundamentally guided by their functional and architectural affinity to MobileNetV3, which was originally designated in the initial project schedule of the internship and for which official pre-trained weights were readily accessible. Beyond sharing key lightweight design principles with MobileNetV3, the experimental objective was to explore and benchmark a heterogeneous cross-section of modern computer vision paradigms. These four models were selected after evaluating 20–25 other models, as they achieved the best overall results after distillation.

- Simple_vits (vit_tiny_patch16) → light version of the classical ViT
- MobileNetV3-Large (mobilenetv3_large_100) → convolutional model
- MobileViT (mobilevit_xs) → lightweight hybrid vision-transformer
- GhostNet (ghostnet_100) → convolutional networks with ghost blocks

This diversity allowed the distillation pipeline to assess how distinct structural mechanisms—spanning depthwise convolutions, linear feature redundancy, and self-attention operations—respond to regression-based knowledge transfer when trained under identical teacher supervision.

6.2 Data Organization and Logit Caching

Because iterative distillation over large-scale facial datasets introduces substantial redundant computation if the teacher is queried online at every epoch, a structured data preparation and caching pipeline was implemented. To structure the training and validation workflows reliably, an **80–20 train-test data split** was adopted.

This distribution was specifically chosen because the 80/20 ratio represents one of the standard, official benchmark evaluation splits defined for the MORPH2 dataset in the literature, thereby ensuring consistent data partitioning across experiments and establishing fair comparability against established baselines.

Prior to student training, an offline extraction script (`extract_teacher_logits.py`) iterated through all training entries cataloged in a self-made log, `distillation.json`. **The frozen teacher network processed each preprocessed face crop and stored the corresponding output representations directly into a persistent directory cache (`teacher_logits_cache`)** serialized as PyTorch tensor files in `.pt` format. In this context, logits represent the raw, unnormalized continuous numeric outputs generated by the final projection layers of the neural network prior to the application of bounding, activation, or probability scaling functions.

During distillation epochs, a custom dataset class (`Morph2UTKDataset`) loaded the input images alongside their cached teacher logits. Decoupling teacher forward passes from student gradient updates **eliminated GPU memory bottlenecks** and accelerated overall training throughput.

6.3 Two-Phase Training Protocol

To bridge representation disparities between heterogeneous student topologies and the teacher network, the distillation procedure was structured into two consecutive phases:

Phase 1: Gender-Based Student Pre-Conditioning (1-2 epoch)

In the first training stage, the student network is pre-conditioned using supervised gender classification. During this phase, only the gender prediction head is explicitly included in the optimization objective, using a standard cross-entropy loss between the student predictions and the ground-truth gender labels. Although the complete student network is updated through backpropagation, the age regression output and the teacher logits do not contribute to the loss at this stage. This initial supervised training allows the student to learn stable facial representations from the target dataset before introducing the more complex multi-task distillation objective.

Phase 2: Supervised Multi-Task Knowledge Distillation (3-6 epoch)

In the second stage, the complete student network is trained end-to-end using the proposed `DistillationLoss`. The objective jointly optimizes age regression and gender classification while incorporating the cached logits produced by the teacher model. The age task uses an MSE-based regression loss, whereas gender

classification is optimized using cross-entropy. Both losses are computed on an instance-wise basis, allowing the teacher contribution to be selectively applied for each training sample.

A **teacher-bounded strategy** is employed to prevent the student from learning from inaccurate teacher predictions. For each sample, the teacher’s prediction error is compared with the student’s error relative to the ground-truth label. Knowledge transfer is activated only when the teacher provides a lower error than the student. Consequently, the student retains its supervised learning signal for all samples while receiving additional guidance from the teacher only when the teacher prediction is more reliable. The final objective combines the supervised student loss and the bounded distillation loss using a weighting factor of $\alpha = 0.5$, providing an equal contribution from both sources of supervision.

Unlike conventional knowledge distillation approaches that employ temperature-scaled soft targets, the proposed implementation directly operates on the teacher logits using a temperature of $T=1$. Instead of softening the teacher distribution, knowledge transfer is selectively activated through the teacher-bounded loss.

6.4 Performance Results

To rigorously assess generalizability and domain transfer, models were trained under two distinct supervision regimes:

Weights Distilled on the 80% UTKFace Training Set: Evaluated on the 20% UTKFace test set (5,115 images) and cross-evaluated on the 20% MORPH2 test set (10,940 images).

Weights Distilled on the 80% MORPH2 Training Set: Evaluated on the 20% UTKFace test set (5,115 images) and the 20% MORPH2 test set (10,940 images).

All metrics were computed across 5 benchmark iterations using batch size 1 on CUDA

Table 3: Evaluation of student architectures distilled on the 80% UTKFace training set (averaged over 5 iterations, batch size = 1).

Metric	Baseline	Simple_vit	Vits_xs	Mobile_100	Ghostnet
<i>Evaluated on 20% UTK (5,115 photos)</i>					
MAE (years) ↓	5.54	20.88	7.72	8.31	8.76
RMSE ↓	7.54	23.61	10.36	11.00	11.50
FPS ↑	40.67	91.05	58.06	80.19	54.18
Inference Time (s) ↓	125.76	56.20	88.10	63.79	94.45
VRAM (MB) ↓	115.77	32.14	30.81	32.88	30.45
Model Size (MB) ↓	98.70	21.19	7.63	16.56	15.48
Gender Acc. (%) ↑	91.72%	86.40%	90.04%	90.28%	87.91%
<i>Cross-evaluated on 20% MORPH2 (10,940 photos)</i>					
MAE (years) ↓	4.45	21.42	4.45	4.46	4.46
RMSE ↓	5.59	23.80	5.59	5.59	5.59
FPS ↑	40.59	91.44	56.88	77.60	53.90
Inference Time (s) ↓	270.00	119.73	192.37	141.00	203.45
VRAM (MB) ↓	115.77	32.14	30.81	32.88	30.45
Model Size (MB) ↓	98.70	21.19	7.62	16.56	15.48
Gender Acc. (%) ↑	97.29%	84.84%	97.29%	97.29%	97.29%

Table 4: Evaluation of student architectures distilled on the 80% MORPH2 training set (averaged over 5 iterations, batch size = 1).

Metric	Baseline	Simple_vit	Vits_xs	Mobile_100	Ghostnet
<i>Cross-evaluated on 20% UTK (5,115 photos)</i>					
MAE (years) ↓	5.54	21.02	14.37	14.09	15.77
RMSE ↓	7.54	26.67	17.67	18.01	19.44
FPS ↑	40.67	89.71	59.13	82.12	52.72
Inference Time (s) ↓	125.76	57.08	86.67	62.40	97.16
VRAM (MB) ↓	115.77	32.14	30.81	32.88	30.45
Model Size (MB) ↓	98.70	21.19	7.63	16.57	15.49
Gender Acc. (%) ↑	91.72%	64.26%	78.10%	78.58%	78.21%
<i>Evaluated on 20% MORPH2 (10,940 photos)</i>					
MAE (years) ↓	4.45	17.27	4.53	4.43	4.22
RMSE ↓	5.59	18.88	5.76	5.65	5.39
FPS ↑	40.59	87.67	60.69	79.58	53.37
Inference Time (s) ↓	270.00	124.90	180.55	137.61	205.03
VRAM (MB) ↓	115.77	32.14	30.81	32.88	30.45
Model Size (MB) ↓	98.70	21.19	7.63	16.57	15.49
Gender Acc. (%) ↑	97.29%	97.44%	99.27%	99.21%	99.28%

6.5 Performance Analysis and Key Findings

Across all configurations, distillation achieved dramatic reductions in storage footprint and hardware resource utilization. Compared to the 98.70 MB footprint and 115.77 MB peak VRAM allocation of the baseline teacher, all student backbones compressed the parameter storage to between 7.62 MB and 21.19 MB (delivering an **78.5% to 92.3% size reduction**) while capping operational VRAM below 33 MB (a **~72% to 73% runtime memory drop**).

In terms of execution speed, every student variant outpaced the baseline (40.6 FPS). SimpleViT achieved the highest raw throughput, consistently exceeding 87–91 FPS (more than $2.2\times$ baseline speed), though it exhibited representation collapse on continuous age regression (MAEs exceeding 17–21 years). In contrast, MobileNetV3-Large (Mobile_100) struck the strongest balance between throughput and regression precision, reaching 77.60–82.12 FPS (**a ~91% to 102% throughput speedup**) across all test sets while preserving low regression error.

Students distilled on the UTKFace training set retained the teacher’s precision nearly intact on MORPH2: Vits_xs matched the baseline MAE of 4.45 years with an RMSE of 5.59 and 97.29% gender accuracy, while Mobile_100 and Ghostnet trailed by only 0.01 years (MAE 4.46).

When **distilled directly on the MORPH2** training set and also tested, both convolutional students outperformed the teacher on in-domain test data: Mobile_100 lowered MAE to 4.43 years, while GhostNet delivered the best overall regression accuracy with an MAE of 4.22 years and an RMSE of 5.39 (outperforming the baseline’s 4.45 MAE and 5.59 RMSE). Moreover, Vits_xs, Mobile_100, and Ghostnet **all surpassed the teacher’s gender accuracy, reaching 99.27%, 99.21%, and 99.28%** respectively.

However, **cross-domain evaluation exposed clear generalization boundaries**: while models distilled on UTKFace transferred smoothly to MORPH2 (yielding competitive MAEs around 4.45–4.46 years), models distilled strictly on MORPH2 degraded significantly when deployed to the more heterogeneous UTKFace test set (MAEs increasing to 14.09–15.77 years and gender accuracy dropping to 78%).

The **performance disparities observed during cross-dataset evaluation are directly linked to the structural and demographic characteristics of the benchmarks**. UTKFace is characterized by broad demographic variability, covering an age span from 0 to 100 years and incorporating diverse ethnic distributions across unconstrained conditions. Conversely, MORPH2 consists of booking and custody portraits limited to a narrower age bracket of 16 to 77 years, with repeated longitudinal captures of the same individuals under uniform lighting, fixed frontal angles, and controlled backgrounds. Furthermore, the baseline MiVOLO-D1 teacher was

not pre-trained on a balanced or complete 0–100 continuous age spectrum. Because student backbones optimize their feature spaces against teacher supervision rather than direct ground truth alone, their generalization boundaries remain coupled to the distribution in which the distillation occurs.

7 Future work

1. Beyond the primary PTQ and logit-based distillation pathways evaluated here, further research should implement and benchmark complementary acceleration techniques reviewed in the theoretical framework
2. The distillation experiments conducted during this project operated under constrained training schedules. A systematic exploration of extended epoch budgets, convergence decay curves, and adaptive learning rate schedulers across both distillation phases is needed to evaluate whether lightweight backbones can recover higher regression fidelity without overfitting to the supervisory split.
3. While the current distillation objective directly utilized raw teacher logits and instance-level reductions, incorporating a tunable distillation temperature (T) parameter into the classification loss represents a key refinement.
4. The experimental scope of this study was strictly constrained to face-only processing using MiVOLO-D1, deliberately bypassing person-body detections. Future work should expand the pipeline to evaluate MiVOLO-D2, ingesting both facial and full-body crops simultaneously. Benchmarking this dual-crop input modality will determine how much supplementary contextual information improves demographic estimation and whether full-body features can be effectively transferred into compact student models.